《Writing effective tools for agents — with agents》精读笔记

> 原文：Anthropic Engineering Blog，Ken Aizawa 等，2025-09-11。 本笔记为中文学习笔记，以要点归纳和个人解读为主，不是原文译文；术语、结论与细节请以原文为准。

一句话结论

给 Agent 用的工具不是"给人用的 API 换个壳"。它是确定性系统和非确定性 Agent 之间的一份新契约，要按 Agent 的认知方式来设计，并且用评测驱动的方式迭代，最好让 Agent 自己参与优化。

1. 工具到底是什么

- 传统函数调用是"确定性对确定性"：同样的输入永远走同样的路径。
- Agent 调工具时，同一句用户请求可能触发调用工具、凭常识直接回答、先反问澄清等多种行为，还可能误用工具或产生幻觉。
- 因此作者主张：不要像给其他开发者写 SDK 那样写 MCP 工具，而要为 Agent 设计。一个有意思的观察是：对 Agent 最"顺手"的工具，往往对人也很直观。

2. 写工具的闭环流程

作者给出的是一个"原型 → 评测 → 协作改进"的循环。

2.1 快速做原型

- 用 Claude Code 写工具时，把依赖库 / API / SDK 的文档一并喂给模型，很多官方站点提供扁平化的 `llms.txt`。
- 把工具包成本地 MCP 服务器或桌面扩展（DXT），在 Claude Code / Claude Desktop 里直接试用；也可以通过 API 直接传入工具做程序化测试。
- 自己先上手用一遍，再收集用户反馈，建立对真实用例的直觉。

2.2 生成评测任务

- 任务应来自真实场景、基于真实数据源，避免过于简单的"沙盒"。
- 好任务往往需要多次甚至几十次工具调用。作者对比了"强任务"和"弱任务"：强任务描述的是业务目标（例如"排查某客户被重复扣费，并看看还有没有其他客户受影响"），弱任务则几乎把调用参数直接告诉了模型。
- 每个任务都要配一个可验证的结果。验证器可以是字符串比对，也可以让模型做裁判，但不要因为格式、标点、同义表述这类无关差异误判。
- 可以标注"期望调用的工具"，但不要过度规定解题路径，因为正确路径可能不止一条。

2.3 跑评测

- 推荐直接用 LLM API 写简单的 agentic 循环：一个 while 循环，交替调用模型和工具，每个任务一个循环。
- 让评测 Agent 在输出结构化答案之外，也输出推理和反馈块。先写推理再调工具，相当于触发思维链；在 Claude 上可以直接开 interleaved thinking。
- 除了准确率，还应收集：单次调用和整个任务的耗时、工具调用总次数、token 消耗、工具报错。调用轨迹能暴露 Agent 的常见工作流，从而找到可以合并的工具。

2.4 分析结果

- Agent 是发现问题的好帮手，但"它没说的"往往比"它说的"更重要。
- 要读推理过程和原始轨迹（含工具请求与响应），读出言外之意。
- 看指标：大量重复调用可能说明分页或截断参数需要调整；大量参数错误可能说明描述不清或缺示例。
- 实例：Claude 的 web search 工具上线时，模型会莫名在 query 里追加年份 "2025"，导致结果偏差；最后是通过改工具描述纠正的。

2.5 让 Agent 帮你改工具

- 把评测轨迹拼起来交给 Claude Code，让它批量分析和重构工具，并保持实现与描述前后一致。
- 文中大部分建议本身就来自这种反复优化。作者用留出测试集防止对"训练用评测"过拟合，结果发现还能超过研究员手写或模型生成的"专家版"工具。

3. 写好工具的五条原则

3.1 选对要做的工具（也决定不做哪些）

- 工具不是越多越好。常见错误是把现有 API 端点一一包装成工具。
- 关键约束：Agent 的上下文有限，而计算机内存便宜。让 Agent 逐 token 读完"全部联系人"，相当于逐页翻通讯录做暴力搜索。
- 建议从少量、面向高价值工作流的工具起步，与评测任务对齐后再扩展。
- 工具可以在内部合并多个操作，典型改造思路：
  - `list_users` + `list_events` + `create_event` → 一个 `schedule_event`（找空档并预约）
  - `read_logs` → `search_logs`（只返回相关行和少量上下文）
  - `get_customer_by_id` + `list_transactions` + `list_notes` → `get_customer_context`
- 每个工具目的要清晰、互不重叠；工具的拆分方式应接近人拿着同样资源时会怎么拆解任务。

3.2 命名空间

- Agent 可能同时接入几十个 MCP 服务器、上百个工具，功能重叠或含义模糊时容易选错。
- 用公共前缀分组，例如按服务（`asana_search` / `jira_search`）或按资源

（`asana_projects_search` / `asana_users_search`）。

- 前缀式和后缀式命名对评测结果有不可忽略的影响，且因模型而异，应以自己的评测为准。
- 好的拆分同时减少了进入上下文的工具描述数量，也把部分"计算"从上下文里挪回工具内部。

3.3 返回有意义的上下文

- 只返回高信号信息：优先返回 `name`、`image_url`、`file_type` 这类能直接指导下一步的字段，少返回 `uuid`、`mime_type` 这类底层标识。
- 把随机字母数字 ID 解析成有语义的名字（甚至用从 0 开始的序号）能明显提升检索精度、减少幻觉。
- 需要同时支持"给模型看"和"给下一个工具用"时，可以暴露一个 `response_format` 枚举参数（如 `concise` / `detailed`），思路类似 GraphQL 的按需取字段。文中的 Slack 例子里，简洁模式的 token 大约是详细模式的三分之一。
- 返回结构用 XML、JSON 还是 Markdown 也会影响效果，没有通用最优，靠评测决定。

3.4 控制返回的 token 量

- 对可能很大的响应，组合使用分页、范围选择、过滤、截断，并给出合理默认值。Claude Code 默认把单次工具响应限制在 25,000 token。
- 截断时要附带引导语，例如建议 Agent 改用多次小而精准的搜索。
- 报错信息也要做"提示工程"：给出具体、可执行的修改建议，而不是不透明的错误码或堆栈。

3.5 对工具描述做提示工程

- 这是性价比最高的手段之一，因为工具描述会整体进入上下文。
- 写描述时，想象是在给新同事讲这个工具：把你默认知道的隐含知识（查询格式、术语、资源间关系）写明白。
- 输入输出要无歧义，并用严格的数据模型约束；参数名要具体，例如用 `user_id` 而不是 `user`。
- 作者举例：精细打磨工具描述后，Claude Sonnet 3.5 在 SWE-bench Verified 上拿到了当时的最佳成绩。
- MCP 场景下可以用 tool annotations 标明哪些工具需要开放世界访问、哪些会产生破坏性修改。

4. 展望

有效的工具有几个共同特征：定义清晰、有意为之；节约使用上下文；能在多种工作流中组合；让 Agent 能直观解决真实任务。协议和模型会继续演进，但只要坚持评测驱动，工具就能跟着 Agent 一起进化。

5. 对本仓库 web-search-agent-go 的启示（个人解读）

- 失败文案即提示：`internal/contract` 里的失败常量相当于工具的"报错响应"，可以按 3.4 节

的思路写成可执行建议（如"请缩小查询范围"），而不是只报"失败"。

- 评测已经具备雏形：`internal/acceptance` 的 18 项断言加黄金回放，本质就是文中说的"可验证结果 + 程序化循环"。可以再补上工具调用次数、token、耗时等指标。
- 警惕 query 污染：文中"自动追加 2025"的例子提醒我们，检查模型生成的搜索词里有没有擅自加日期，可以在验收中加一条断言。
- 返回精简：web search 的 observation 可以考虑提供简洁 / 详细两种格式，降低多轮 ReAct 的上下文占用。

6. 原文章节对照

原文章节	本笔记位置
What is a tool?	第 1 节
How to write tools（Prototype / Evaluation / Collaborating）	第 2 节
Principles for writing effective tools	第 3 节
Looking ahead	第 4 节